

Chapter 6. Agent Architecture

Building on the architectures described in [Chapter 5](#), this chapter will cover what is perhaps the most important of all current LLM architectures, the agent architecture. First, we introduce what makes LLM agents unique, then we show how to build them and how to extend them for common use cases.

In the artificial intelligence field, there is a long history of creating (intelligent) agents, which can be most simply defined as “something that acts,” in the words of Stuart Russell and Peter Norvig in their *Artificial Intelligence* (Pearson, 2020) textbook. The word *acts* actually carries a little more meaning than meets the eye:

- Acting requires some capacity for deciding what to do.
- Deciding what to do implies having access to more than one possible course of action. After all, a decision without options is no decision at all.
- In order to decide, the agent also needs access to information about the external environment (anything outside of the agent itself).

So an *agentic* LLM application must be one that uses an LLM to pick from one or more possible courses of action, given some context about the current state of the world or some desired next state. These attributes are usually implemented by mixing two prompting techniques we first met in the [Preface](#):

Tool calling

Include a list of external functions that the LLM can make use of in your prompt (that is, the actions it can decide to take) and provide instructions on how to format its choice in the output it generates. You'll see in a moment what this looks like in the prompt.

Chain-of-thought

Researchers have found that LLMs “make better decisions” when given instructions to reason about complex problems by breaking them down into granular steps to be taken in sequence. This is usually done either by adding instructions along the lines of “think step by step” or including examples of questions and their decomposition into several steps/actions.

Here's an example prompt using both tool calling and chain-of-thought:

Tools:

search: this tool accepts a web search query and returns the top results.

calculator: this tool accepts math expressions and returns their result.

If you want to use tools to arrive at the answer, output the list of tools and inputs in CSV format, with the header row: tool,input.

Think step by step; if you need to make multiple tool calls to arrive at the answer, return only the first one.

How old was the 30th president of the United States when he died?

tool,input

And the output, when run against `gpt-3.5-turbo` at temperature 0 (to ensure the LLM follows the desired output format, CSV) and newline as the stop sequence (which instructs the LLM to stop producing output when it reaches this character). This makes the LLM produce a single action (as expected, given the prompt asked for this):

search,30th president of the United States

The most recent LLMs and chat models have been fine-tuned to improve their performance for tool-calling and chain-of-thought applications, removing the need for adding specific instructions to the prompt:

add example prompt and output for tool-calling model

The Plan-Do Loop

What makes the agent architecture different from the architectures discussed in [Chapter 5](#) is a concept we haven't covered yet: the LLM-driven loop.

Every programmer has encountered loops in their code before. By *loop*, we mean running the same code multiple times until a stop condition is hit. The key to the agent architecture is to have an LLM control the stop condition—that is, decide when to stop looping.

What we'll run in this loop will be some variation of the following:

- Planning an action or actions
- Executing said action(s)

Picking up on the example in the previous section, we'll next run the

`search` tool with the input `30th president of the United`

states, which produces this output:

```
Calvin Coolidge (born John Calvin Coolidge Jr.; /'ku:lɪdʒ/; July 4, 1872 – January 5, 1933) was an American attorney and politician who served as the 30th president of the United States from 1923 to 1929. John Calvin Coolidge Jr.
```

And then we'll rerun the prompt, with a small addition:

```
Tools:
search: this tool accepts a web search query and returns the top results.
calculator: this tool accepts math expressions and returns their result.
output: this tool ends the interaction. Use it when you have the final answer.

If you want to use tools to arrive at the answer, output the list of tools and their
inputs in CSV format, with this header row: tool,input

Think step by step; if you need to make multiple tool calls to arrive at
the answer, return only the first one.

How old was the 30th president of the United States when he died?

tool,input

search,30th president of the United States

search: Calvin Coolidge (born John Calvin Coolidge Jr.; /'ku:lɪdʒ/; July 4, 1872 – January 5, 1933) was an American attorney and politician who served as the 30th president of the United States from 1923 to 1929. John Calvin Coolidge Jr.

tool,input
```

And the output:

```
calculator,1933 - 1872
```

Notice we added two things:

- An “output” tool—which the LLM should use when it has found the final answer, and which we'd use as the signal to stop the loop.
- The result of the tool call from the preceding iteration, simply with the name of the tool and its (text) output. This is included in order to allow the LLM to move on to the next step in the interaction. In other words, we're telling the LLM, “Hey, we got the results you asked for, what do you want to do next?”

Let's continue with a third iteration:

Tools:

search: this tool accepts a web search query and returns the top results.

calculator: this tool accepts math expressions and returns their result.

If you want to use tools to arrive at the answer, output the list of tools and inputs in CSV format, with this header row: tool,input.

output: this tool ends the interaction. Use it when you have the final answer.

Think step by step; if you need to make multiple tool calls to arrive at the answer, return only the first one.

How old was the 30th president of the United States when he died?

tool,input

search,30th president of the United States

search: Calvin Coolidge (born John Calvin Coolidge Jr.; /'ku:lɪdʒ/; July 4, January 5, 1933) was an American attorney and politician who served as the 30th president of the United States from 1923 to 1929. John Calvin Coolidge Jr.

tool,input

calculator,1933-1872

calculator: 61

tool, input

And the output:

output, 61

With the result from the `calculator` tool, the LLM now has enough information to provide the final answer, so it picked the `output` tool and chose “61” as the final answer.

This is what makes the agent architecture so useful—the LLM is given the agency to decide. The next step is to arrive at an answer and decide how many steps to take—that is, when to stop.

This architecture, called [ReAct](#), was first proposed by Shunyu Yao et al. The rest of this chapter explores how to improve the performance of the agent architecture, motivated by the email assistant example from [Chapter 5](#).

But first, let’s see what it looks like to implement the basic agent architecture using a chat model and LangGraph.

Building a LangGraph Agent

For this example, we need to install additional dependencies for the search tool we chose to use, DuckDuckGo. To install it for Python:

Python

```
pip install duckduckgo-search
```

And for JS, we also need to install a dependency for the calculator tool:

JavaScript

```
npm i duck-duck-scraper expr-eval
```

With that complete, let's get into the actual code to implement the agent architecture:

Python

```
import ast
from typing import Annotated, TypedDict

from langchain_community.tools import DuckDuckGoSearchRun
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI

from langgraph.graph import START, StateGraph
from langgraph.graph.message import add_messages
from langgraph.prebuilt import ToolNode, tools_condition

@tool
def calculator(query: str) -> str:
    """A simple calculator tool. Input should be a mathematical expression.
    return ast.literal_eval(query)

search = DuckDuckGoSearchRun()
tools = [search, calculator]
model = ChatOpenAI(temperature=0.1).bind_tools(tools)

class State(TypedDict):
    messages: Annotated[list, add_messages]

def model_node(state: State) -> State:
    res = model.invoke(state["messages"])
    return {"messages": res}

builder = StateGraph(State)
builder.add_node("model", model_node)
```

```

builder.add_node("tools", ToolNode(tools))
builder.add_edge(START, "model")
builder.add_conditional_edges("model", tools_condition)
builder.add_edge("tools", "model")

graph = builder.compile()

```

JavaScript

```

import {
  DuckDuckGoSearch
} from "@langchain/community/tools/duckduckgo_search";
import {
  Calculator
} from "@langchain/community/tools/calculator";
import {
  StateGraph,
  Annotation,
  messagesStateReducer,
  START,
} from "@langchain/langgraph";
import {
  ToolNode,
  toolsCondition
} from "@langchain/langgraph/prebuilt";

const search = new DuckDuckGoSearch();
const calculator = new Calculator();
const tools = [search, calculator];
const model = new ChatOpenAI({
  temperature: 0.1
}).bindTools(tools);

const annotation = Annotation.Root({
  messages: Annotation({
    reducer: messagesStateReducer,
    default: () => []
  }),
});

async function modelNode(state) {
  const res = await model.invoke(state.messages);
  return { messages: res };
}

const builder = new StateGraph(annotation)
  .addNode("model", modelNode)
  .addNode("tools", new ToolNode(tools))
  .addEdge(START, "model")
  .addConditionalEdges("model", toolsCondition)
  .addEdge("tools", "model");

const graph = builder.compile();

```

The visual representation is shown in [Figure 6-1](#).

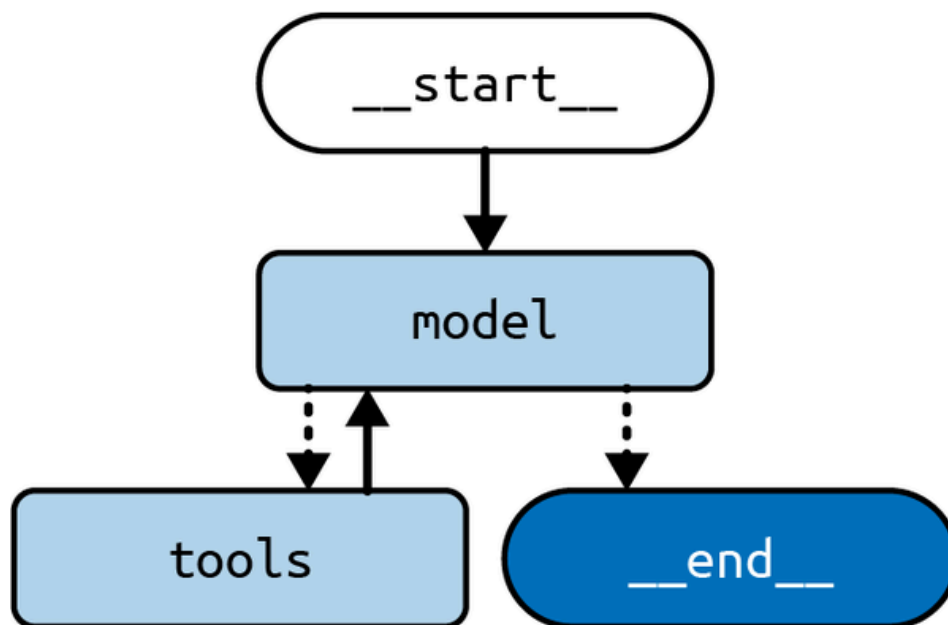


Figure 6-1. The agent architecture

A few things to notice here:

- We're using two tools in this example: a search tool and a calculator tool, but you could easily add more or replace the ones we used. In the Python example, you also see an example of creating a custom tool.
- We've used two convenience functions that ship with LangGraph. `ToolNode` serves as a node in our graph; it executes the tool calls requested in the latest AI message found in the state and returns a `ToolMessage` with the results of each. `ToolNode` also handles exceptions raised by tools—using the error message to build a `ToolMessage` that is then passed to the LLM—which may decide what to do with the error.
- `tools_condition` serves as a conditional edge function that looks at the latest AI message in the state and routes to the `tools` node if there are any tools to execute. Otherwise, it ends the graph.
- Finally, notice that this graph loops between the model and tools nodes. That is, the model itself is in charge of deciding when to end the computation, which is a key attribute of the agent architecture. Whenever we code a loop in LangGraph, we'll likely want to use a conditional edge, as that allows you to define the *stop condition* when the graph should exit the loop and stop executing.

Now let's see how it does in the previous example:

Python

```
input = {  
    "messages": [  
        HumanMessage("""How old was the 30th president of the United States  
when he died?""")
```

```

    ]
}
for c in graph.stream(input):
    print(c)

```

JavaScript

```

const input = {
  messages: [
    HumanMessage(`How old was the 30th president of the United States when l
      died?`)
  ]
}
for await (const c of await graph.stream(input)) {
  console.log(c)
}

```

The output:

```

{
  "model": {
    "messages": AIMessage(
      content="",
      tool_calls=[
        {
          "name": "duckduckgo_search",
          "args": {
            "query": "30th president of the United States age at
              death"
          },
          "id": "call_ZWRbPmjvo0fYkwyo4HCYUsar",
          "type": "tool_call",
        }
      ],
    )
  }
}
{
  "tools": {
    "messages": [
      ToolMessage(
        content="Calvin Coolidge (born July 4, 1872, Plymouth, Vermo
          U.S.—died January 5, 1933, Northampton, Massachusetts) v
          the 30th president of the United States (1923–29). Cool:
          acceded to the presidency after the death in office of
          Warren G. Harding, just as the Harding scandals were cor
          to light....",
        name="duckduckgo_search",
        tool_call_id="call_ZWRbPmjvo0fYkwyo4HCYUsar",
      )
    ]
  }
}

```



```

    }
    {
        "model": {
            "messages": AIMessage(
                content="Calvin Coolidge, the 30th president of the United States,
                    died on January 5, 1933, at the age of 60.",
            )
        }
    }

```

Walking through this output:

1. First the `model` node executed and decided to call the `duckduckgo_search` tool, which led the conditional edge to route us to the `tools` node after.
2. The `ToolNode` executed the search tool and got the search results printed above, which actually contain the answer “Age and Year of Death . January 5, 1933 (aged 60)”.
3. The `model` tool was called again, this time with the search results as the latest message, and produced the final answer (with no more tool calls); therefore, the conditional edge ended the graph.

Next, let’s look at a few useful extensions to this basic agent architecture, customizing both planning and tool calling.

Always Calling a Tool First

In the standard agent architecture, the LLM is always called upon to decide what tool to call next. This arrangement has a clear advantage: it gives the LLM ultimate flexibility to adapt the behavior of the application to each user query that comes in. But this flexibility comes at a cost: unpredictability. If, for instance, you, the developer of the application, know that the search tool should always be called first, that can actually be beneficial to your application:

1. It will reduce overall latency, as it will skip the first LLM call that would generate that request to call the search tool.
2. It will prevent the LLM from erroneously deciding it doesn’t need to call the search tool for some user queries.

On the other hand, if your application doesn’t have a clear rule of the kind “you should always call this tool first,” introducing such a constraint would actually make your application worse.

Let’s see what it looks like to do this:

Python

```

import ast
from typing import Annotated, TypedDict
from uuid import uuid4

from langchain_community.tools import DuckDuckGoSearchRun
from langchain_core.messages import AIMessage, HumanMessage, ToolCall
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI

from langgraph.graph import START, StateGraph
from langgraph.graph.message import add_messages
from langgraph.prebuilt import ToolNode, tools_condition

@tool
def calculator(query: str) -> str:
    """A simple calculator tool. Input should be a mathematical expression.
    return ast.literal_eval(query)

search = DuckDuckGoSearchRun()
tools = [search, calculator]
model = ChatOpenAI(temperature=0.1).bind_tools(tools)

class State(TypedDict):
    messages: Annotated[list, add_messages]

def model_node(state: State) -> State:
    res = model.invoke(state["messages"])
    return {"messages": res}

def first_model(state: State) -> State:
    query = state["messages"][-1].content
    search_tool_call = ToolCall(
        name="duckduckgo_search", args={"query": query}, id=uuid4().hex
    )
    return {"messages": AIMessage(content="", tool_calls=[search_tool_call])

builder = StateGraph(State)
builder.add_node("first_model", first_model)
builder.add_node("model", model_node)
builder.add_node("tools", ToolNode(tools))
builder.add_edge(START, "first_model")
builder.add_edge("first_model", "tools")
builder.add_conditional_edges("model", tools_condition)
builder.add_edge("tools", "model")

graph = builder.compile()

```

JavaScript

```

import {
  DuckDuckGoSearch
} from "@langchain/community/tools/duckduckgo_search";
import {

```

```

    Calculator
  } from "@langchain/community/tools/calculator";
import {
  AIMessage,
} from "@langchain/core/messages";
import {
  StateGraph,
  Annotation,
  messagesStateReducer,
  START,
} from "@langchain/langgraph";
import {
  ToolNode,
  toolsCondition
} from "@langchain/langgraph/prebuilt";

const search = new DuckDuckGoSearch();
const calculator = new Calculator();
const tools = [search, calculator];
const model = new ChatOpenAI({ temperature: 0.1 }).bindTools(tools);

const annotation = Annotation.Root({
  messages: Annotation({ reducer: messagesStateReducer, default: () => [] }
});

async function firstModelNode(state) {
  const query = state.messages[state.messages.length - 1].content;
  const searchToolCall = {
    name: "duckduckgo_search",
    args: { query },
    id: Math.random().toString(),
  };
  return {
    messages: [new AIMessage({ content: "", tool_calls: [searchToolCall] })]
  };
}

async function modelNode(state) {
  const res = await model.invoke(state.messages);
  return { messages: res };
}

const builder = new StateGraph(annotation)
  .addNode("first_model", firstModelNode)
  .addNode("model", modelNode)
  .addNode("tools", new ToolNode(tools))
  .addEdge(START, "first_model")
  .addEdge("first_model", "tools")
  .addEdge("tools", "model")
  .addConditionalEdges("model", toolsCondition);

const graph = builder.compile();

```

The visual representation is shown in [Figure 6-2](#).

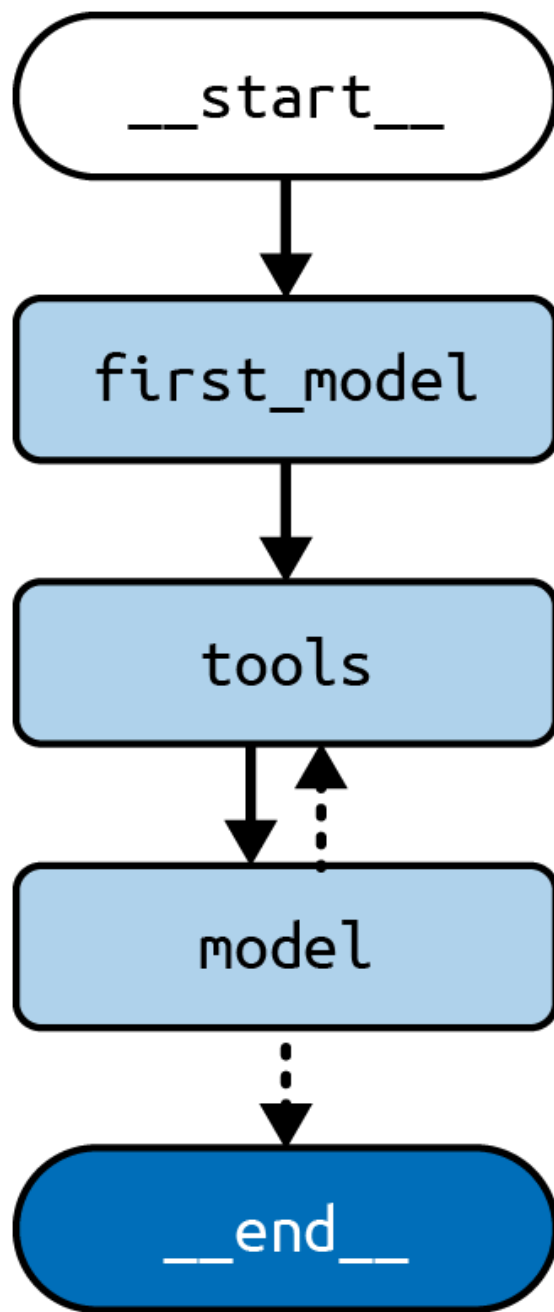


Figure 6-2. Modifying the agent architecture to always call a specific tool first

Notice the differences compared to the previous section:

- Now, we start all invocations by calling `first_model`, which doesn't call an LLM at all. It just creates a tool call for the search tool, using the user's message verbatim as the search query. The previous architecture would have the LLM generate this tool call (or some other response it deemed better).
- After that, we proceed to `tools`, which is identical to the previous example, and from there we proceed to the `agent` node as before.

Now let's see some example output, for the same query as before:

Python

```
input = {
    "messages": [
        HumanMessage("""How old was the 30th president of the United States
when he died?""")
```

```

    ]
}
for c in graph.stream(input):
    print(c)

```

JavaScript

```

const input = {
  messages: [
    HumanMessage(`How old was the 30th president of the United States when l
      died?`)
  ]
}
for await (const c of await graph.stream(input)) {
  console.log(c)
}

```

The output:

```

{
  "first_model": {
    "messages": AIMessage(
      content="",
      tool_calls=[
        {
          "name": "duckduckgo_search",
          "args": {
            "query": "How old was the 30th president of the United
              States when he died?"
          },
          "id": "9ed4328dcdea4904b1b54487e343a373",
          "type": "tool_call",
        }
      ],
    )
  }
}
{
  "tools": {
    "messages": [
      ToolMessage(
        content="Calvin Coolidge (born July 4, 1872, Plymouth, Vermont;
          U.S.—died January 5, 1933, Northampton, Massachusetts) was
          the 30th president of the United States (1923–29). Coolidge
          acceded to the presidency after the death in office of Warren
          G. Harding, just as the Harding scandals were coming
          to light....",
        name="duckduckgo_search",
        tool_call_id="9ed4328dcdea4904b1b54487e343a373",
      )
    ]
  }
}

```

```

    }
    {
        "model": {
            "messages": AIMessage(
                content="Calvin Coolidge, the 30th president of the United States,
                was born on July 4, 1872, and died on January 5, 1933. To
                calculate his age at the time of his death, we can subtract
                birth year from his death year. \n\nAge at death = Death year -
                Birth year\nAge at death = 1933 - 1872\nAge at death = 61
                years\n\nCalvin Coolidge was 61 years old when he died.",
            )
        }
    }
}

```

This time, we skipped the initial LLM call. We first went to `first_model` node, which directly returned a tool call for the search tool. From there we went to the previous flow—that is, we executed the search tool and finally went back to the `model` node to generate the final answer.

Next let's go over what you can do when you have many tools you want to make available to the LLM.

Dealing with Many Tools

LLMs are far from perfect, and they currently struggle more when given multiple choices or excessive information in a prompt. These limitations also extend to the planning of the next action to take. When given many tools (say, more than 10) the planning performance (that is, the LLM's ability to choose the right tool) starts to suffer. The solution to this problem is to reduce the number of tools the LLM can choose from. But what if you do have many tools you want to see used for different user queries?

One elegant solution is to use a RAG step to preselect the most relevant tools for the current query and then feed the LLM only that subset of tools instead of the entire arsenal. This can also help to reduce the cost of calling the LLM (commercial LLMs usually charge based on the length of the prompt and outputs). On the other hand, this RAG step introduces additional latency to your application, so should only be taken when you see performance decreasing after adding more tools.

Let's see how to do this:

Python

```

import ast
from typing import Annotated, TypedDict

```

```

from langchain_community.tools import DuckDuckGoSearchRun
from langchain_core.documents import Document
from langchain_core.messages import HumanMessage
from langchain_core.tools import tool
from langchain_core.vectorstores.in_memory import InMemoryVectorStore
from langchain_openai import ChatOpenAI, OpenAIEmbeddings

from langgraph.graph import START, StateGraph
from langgraph.graph.message import add_messages
from langgraph.prebuilt import ToolNode, tools_condition

@tool
def calculator(query: str) -> str:
    """A simple calculator tool. Input should be a mathematical expression.
    return ast.literal_eval(query)

search = DuckDuckGoSearchRun()
tools = [search, calculator]

embeddings = OpenAIEmbeddings()
model = ChatOpenAI(temperature=0.1)

tools_retriever = InMemoryVectorStore.from_documents(
    [Document(tool.description, metadata={"name": tool.name}) for tool in tools],
    embeddings,
).as_retriever()

class State(TypedDict):
    messages: Annotated[list, add_messages]
    selected_tools: list[str]

def model_node(state: State) -> State:
    selected_tools = [
        tool for tool in tools if tool.name in state["selected_tools"]
    ]
    res = model.bind_tools(selected_tools).invoke(state["messages"])
    return {"messages": res}

def select_tools(state: State) -> State:
    query = state["messages"][-1].content
    tool_docs = tools_retriever.invoke(query)
    return {"selected_tools": [doc.metadata["name"] for doc in tool_docs]}

builder = StateGraph(State)
builder.add_node("select_tools", select_tools)
builder.add_node("model", model_node)
builder.add_node("tools", ToolNode(tools))
builder.add_edge(START, "select_tools")
builder.add_edge("select_tools", "model")
builder.add_conditional_edges("model", tools_condition)
builder.add_edge("tools", "model")

graph = builder.compile()

```

```

import { DuckDuckGoSearch } from "@langchain/community/tools/duckduckgo_search";
import { Calculator } from "@langchain/community/tools/calculator";
import { ChatOpenAI } from "@langchain/openai";
import { OpenAIEmbeddings } from "@langchain/openai";
import { Document } from "@langchain/core/documents";
import { MemoryVectorStore } from "langchain/vectorstores/memory";
import {
  StateGraph,
  Annotation,
  messagesStateReducer,
  START,
} from "@langchain/langgraph";
import { ToolNode, toolsCondition } from "@langchain/langgraph/prebuilt";
import { HumanMessage } from "@langchain/core/messages";

const search = new DuckDuckGoSearch();
const calculator = new Calculator();
const tools = [search, calculator];

const embeddings = new OpenAIEmbeddings();
const model = new ChatOpenAI({ temperature: 0.1 });

const toolsStore = await MemoryVectorStore.fromDocuments(
  tools.map(
    (tool) =>
      new Document({
        pageContent: tool.description,
        metadata: { name: tool.constructor.name },
      })
  ),
  embeddings
);
const toolsRetriever = toolsStore.asRetriever();

const annotation = Annotation.Root({
  messages: Annotation({ reducer: messagesStateReducer, default: () => [] }),
  selected_tools: Annotation(),
});

async function modelNode(state) {
  const selectedTools = tools.filter((tool) =>
    state.selected_tools.includes(tool.constructor.name)
  );
  const res = await model.bindTools(selectedTools).invoke(state.messages);
  return { messages: res };
}

async function selectTools(state) {
  const query = state.messages[state.messages.length - 1].content;
  const toolDocs = await toolsRetriever.invoke(query as string);
  return {
    selected_tools: toolDocs.map((doc) => doc.metadata.name),
  };
}

```



```

};
}

const builder = new StateGraph(annotation)
  .addNode("select_tools", selectTools)
  .addNode("model", modelNode)
  .addNode("tools", new ToolNode(tools))
  .addEdge(START, "select_tools")
  .addEdge("select_tools", "model")
  .addConditionalEdges("model", toolsCondition)
  .addEdge("tools", "model");

```

You can see the visual representation in [Figure 6-3](#).

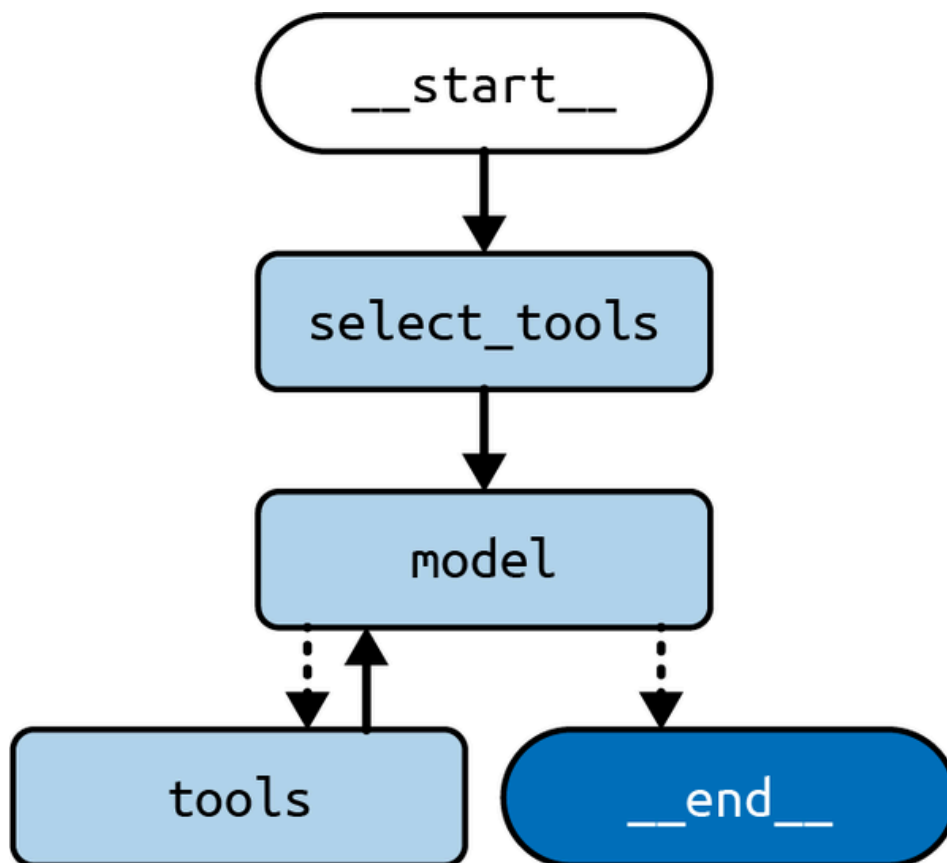


Figure 6-3. Modifying the agent architecture to deal with many tools

NOTE

This is very similar to the regular agent architecture. The only difference is that we stop by the `select_tools` node before entering the actual agent loop. After that, it works just as the regular agent architecture we've seen before.

Now let's see some example output for the same query as before:

Python

```

input = {
  "messages": [
    HumanMessage("How old was the 30th president of the United States when

```

```

        he died?""")
    ]
}
for c in graph.stream(input):
    print(c)

```

JavaScript

```

const input = {
  messages: [
    HumanMessage(`How old was the 30th president of the United States when l
      died?`)
  ]
}
for await (const c of await graph.stream(input)) {
  console.log(c)
}

```

The output:

```

{
  "select_tools": {
    "selected_tools": ['duckduckgo_search', 'calculator']
  }
}
{
  "model": {
    "messages": AIMessage(
      content="",
      tool_calls=[
        {
          "name": "duckduckgo_search",
          "args": {
            "query": "30th president of the United States"
          },
          "id": "9ed4328dcdea4904b1b54487e343a373",
          "type": "tool_call",
        }
      ],
    )
  }
}
{
  "tools": {
    "messages": [
      ToolMessage(
        content="Calvin Coolidge (born July 4, 1872, Plymouth, Vermont; died January 5, 1933, Northampton, Massachusetts) was the 30th president of the United States (1923–29). Coolidge acceded to the presidency after the death in office of Warren G. Harding, just as the Harding scandals were coming to light....",

```

```

        name="duckduckgo_search",
        tool_call_id="9ed4328dcdea4904b1b54487e343a373",
    )
]
}
{
    "model": {
        "messages": AIMessage(
            content="Calvin Coolidge, the 30th president of the United States, was born on July 4, 1872, and died on January 5, 1933. To calculate his age at the time of his death, we can subtract his birth year from his death year. \n\nAge at death = Death year - Birth year\nAge at death = 1933 - 1872\nAge at death = 61 years\n\nCalvin Coolidge was 61 years old when he died.",
        )
    }
}

```

Notice how the first thing we did was query the retriever to get the most relevant tools for the current user query. Then, we proceeded to the regular agent architecture.

Summary

This chapter introduced the concept of *agency* and discussed what it takes to make an LLM application *agentic*: giving the LLM the ability to decide between multiple options by using external information.

We walked through the standard agent architecture built with LangGraph and looked at two useful extensions: how to always call a specific tool first and how to deal with many tools.

[Chapter 7](#) looks at additional extensions to the agent architecture.